Aegis — Project Reference (single source of truth: explains everything + every command)

This is the self-sufficient reference for the whole project — what it does, the RAG backend in detail, every command, evaluation, reset tooling, the unified storage model, the security architecture & threat model, the decisions AND the reasoning. A future agent should be able to continue from this file alone. **If you change something, update this file** (and add a change-log entry at the bottom).

The companion doc is `STATUS_AND_MOCKBANK.md` — what's done, what's left, and the mock-bank integration guide. Two more reference contracts stay standalone: `APISpec.md` (API) and `DatabaseSchema.md` (DB). Frontend lives in `frontend/`.

> This file consolidates the former `RAG_MASTER.md` + `explanation.md` (security) + `SCHEMA_FALLACY_FIX.md` (schema/edge-cases). Last updated: 2026-06-19.

Contents: §0 commands · §1–§16 RAG concept/code/eval · §17 design journey · §19 walkthrough · §20 security architecture & threat model · §21 data schemas & schema-faithful eval · §22 known edge cases & hardening.

Table of contents

0. Quick command reference

Always run from the repo root (`cd C:\Users\nkk77\Desktop\rgbackup\sar-rag_v1`).

All client data lives under one root: `backend/storage/clients/<client_id>/` (`client_0` = dummy; `TEN-xxxx` = real). The eval reads the SAME folders the live system writes.

```
# ---- RETRIEVAL EVAL (deterministic, no Groq) — needs eval.json answer key ----
python eval/ir_metrics.py                 # every client that has an eval.json (=
client_0)
python eval/ir_metrics.py client_0        # one client

# ---- GENERATION EVAL (RAGAS-style, Groq judge + bge) — reference-free, any
client ----
python eval/ragas_eval.py client_0        # the dummy
python eval/ragas_eval.py TEN-0002        # a real client (export its alerts first
↓)

# ---- GENERATION (needs GROQ_API_KEY in backend/.env) ----
# TEST-ONLY demo scripts — always use the dummy client_0's policy + an alert.
python scripts/rag_smoke_test.py          # retrieval only (top-8 + cosine
distances)
python scripts/rag_generate_test.py       # RAG vs no-RAG baseline
python scripts/generate_sar_report.py     # SAR PDF ->
clients/client_0/sar/<id>.pdf (+ JSON in outputs/)
python scripts/demo_full_pipeline.py      # full flow; approved PDF ->
client_0/sar/ (goAML+webhook in outputs/final/)

# ---- DUMMY (client_0): seed / reset ----
python scripts/seed_testing.py            # (re)build client_0 (policy + alerts +
eval.json)
python scripts/build_policy.py            # rebuild ONLY client_0's synthetic
policy PDF
python scripts/reset.py                   # clear runtime (outputs + Chroma);
client data kept
python scripts/reset.py --seed            # clear runtime + rebuild client_0
python scripts/reset.py --barebone        # delete the dummy client_0 (REAL
clients kept)
python scripts/reset.py --barebone --seed # wipe client_0, then rebuild it
python scripts/reset.py --barebone --all-clients  # DANGER: delete EVERY client
(incl. real policies)

# ---- FULL CYCLE: delete -> reseed -> eval (standard sanity check) ----
python scripts/reset.py --barebone --seed # wipe storage/clients/ + rebuild
client_0
python eval/ir_metrics.py                 # IR — expect Recall@8 = 1.0
python eval/ragas_eval.py client_0        # RAGAS (needs the Groq key; makes its
OWN SARs)
python scripts/generate_sar_report.py     # (optional) viewable SAR PDF ->
client_0/sar/; NOT needed for RAGAS
```

```
# ---- REAL client: export its DB alerts to disk for RAGAS ----
python scripts/export_alerts.py TEN-0002 --limit 20           # latest 20: alerts
table -> folder JSON
python scripts/export_alerts.py TEN-0002 --limit 1 --oldest   # just the client's
1st request
python eval/ragas_eval.py TEN-0002                            # then score what
was exported
```

Rule of thumb: IR needs an `eval.json` (so it's `client_0` only); RAGAS needs none (any client). Eval/smoke need no Groq key; generation does.

---

1. What this is

Aegis AML auto-drafts Suspicious Activity Reports (SARs/STRs) for Indian fintechs/brokers. A transaction → normalized → PII-masked → 8 typology rules → if risk ≥ 75, an LLM (Groq) writes a SAR → officer reviews/approves → filed to FIU-India via goAML.

RAG makes that SAR cite the **tenant's actual AML policy** instead of the LLM's general knowledge. It's plain ("vanilla") RAG: parse → chunk → embed → store → cosine-retrieve → inject into the prompt. No reranking/hybrid/critic (we proved with metrics they aren't needed yet).

---

2. Architecture & flow

```
config.py / .env  →  embeddings.py (encoder engine, used by BOTH sides)
   │
   ├ PHASE 1 (index, once per uploaded PDF)
   │   document_ingestion_service.py: parse → sections → chunks → embed →
chroma_client.store
   │
   ├ PHASE 2 (retrieve, per alert)
   │   rag_retrieval_service.py: fired rules → sub-queries → embed →
chroma_client.cosine → top-8
   │
   └ PHASE 3 (generate, per alert)
       llm_agent.py: chunks + data → prompt → Groq → SAR (narrative + structured
JSON)
           → (real) dashboard → officer approves → rehydrate PII → goAML JSON +
PDF → webhook
           → (demo) simulated approval → files in outputs/final/
```

Two **shared modules** (`embeddings.py`, `chroma_client.py`) are called by both the index and retrieve sides, so the encoder and store can never drift apart. **None of retrieval uses an LLM** — bge is an encoder. Groq appears only in Phase 3.

---

3. Backend code (detailed)

All RAG code lives in `backend/app/services/`.

`embeddings.py` — the encoder engine

- Model: `BAAI/bge-small-en-v1.5` via sentence-transformers (SBERT). 384-dim, 512-token max, cosine. Local, free, offline. Provider-swappable (`local`/`openai`).
- Singleton + lazy load (key perf detail):

```
@lru_cache(maxsize=1)
def _local_model():
    from sentence_transformers import SentenceTransformer   # lazy: torch
loads only when used
    return SentenceTransformer(_LOCAL_MODEL_NAME)            # loaded ONCE,
reused
```

- Asymmetric: query gets a prefix, chunks don't:

```
QUERY_INSTRUCTION = "Represent this sentence for searching relevant passages:
"
def embed_documents(texts): return model.encode(texts,
normalize_embeddings=True).tolist()
def embed_query(text):      return model.encode([QUERY_INSTRUCTION+text],
normalize_embeddings=True)[0].tolist()
```

- `count_tokens()` uses bge's own tokenizer so the chunker respects the 512 limit.

`chroma_client.py` — the vector store

```
@lru_cache(maxsize=1)
def get_chroma_client(): return chromadb.PersistentClient(path=_PERSIST_DIR)   #
LOCAL files
def get_tenant_collection(tenant_id):
    return get_chroma_client().get_or_create_collection(
        name=f"tenant_{tenant_id}_docs", metadata={"hnsw:space": "cosine"})    #
per-tenant + cosine
```

`document_ingestion_service.py` — Phase 1 (index)

Constants: `CHUNK_TARGET=350`, `CHUNK_OVERLAP=60`, `MAX_CHUNK_HARD=480`.

- `parse_pdf` — PyMuPDF `get_text("dict")` → keeps font size (heading detection)
  - page number (citations). Body font size = the size covering the most text.
- `_running_lines` — GENERIC running-header/footer detection (lines repeated on ≥50% of pages). No hardcoded client text.
- `split_sections` — split on detected headings; skip page numbers + running furniture; **drop front matter before the first numbered heading** (removes cover bank name/title/doc-control table

generically).

- `_is_heading` — numbered headings always count; non-numbered must be short (≤60 chars) so the document title isn't mistaken for a section.
- `chunk_section` — accumulate whole sentences to ~350 tokens, emit, carry ~60 tokens of overlap into the next chunk; never cut mid-sentence. `_enforce_max_len` hard-splits any single sentence > 480 tokens (prevents silent bge truncation).
- `build_chunks` — prepend `[Context: <doc> - Section: "<heading>"]` to every chunk (so it's findable even without the keyword); attach metadata `{doc_id, filename, section_heading, chunk_index, page_number}`.
- `index_document(tenant_id, chunks)` — `embed_documents` then `collection.add`.

`rag_retrieval_service.py` — Phase 2 (retrieve)

- `RULE_TO_QUERY` — each real `rule_name` → a regulation-worded sub-query.
- We do NOT embed the JSON (masked tokens are semantic noise).
- `build_sub_queries` — one sub-query per fired rule + a fallback from txn type.
- `retrieve_regulatory_context(tenant_id, masked_payload, compliance_results, top_k=8)`:

```
collection = get_tenant_collection(tenant_id)         # ONLY this tenant's
docs
for q in sub_queries:
    res = collection.query(query_embeddings=[embed_query(q)], n_results=5)
    # merge into dict keyed by chunk_id, keep smallest distance (dedupe)
return sorted(merged.values(), key=distance)[:top_k]  # top-8; lower distance
= more relevant
```

`llm_agent.py` — Phase 3 (generate)

- `_format_regulatory_context` — formats the 8 chunks into a REGULATORY CONTEXT block.
- `build_sar_prompt` — injects that block AFTER `<<END DATA>>` (chunks are trusted; keeps them outside the untrusted-payload fence) and pins the JSON schema so `key_indicators` are always `{indicator, regulation, description}` objects.
- `_parse_response` — fence-tolerant (strips ```json fences); a JSON parse failure never discards the narrative (bug we fixed).
- `generate_sar_core(masked_payload, risk_score, model_name, retrieved_chunks)` — DB-free core (so tests run without Postgres). `generate_sar()` is the DB wrapper that persists the SARDraft and rehydrates PII for the bank-facing copy.

---

4. Where ChromaDB stores (LOCAL, no server)

`PersistentClient(path=...)` writes files:

```
<persist_dir>/
├── chroma.sqlite3                 # documents + metadata + ids + catalog
└── <collection-uuid>/             # one folder per collection
```

```
    ├── data_level0.bin            # 384-dim vectors + HNSW graph nodes
    ├── header.bin / length.bin
    └── link_lists.bin             # HNSW graph edges (fast approx-NN search)
```

Default path = `backend/chroma_data/` (gitignored). Test scripts override it to a temp dir so runs don't pollute the repo. Production: point `CHROMA_PERSIST_DIR` at a stable path, or move to Chroma server / pgvector — only that setting changes.

---

5. Multi-tenant isolation

`tenant_id` threads through: `index_document("client_0", …)` → `tenant_client_0_docs`; `retrieve_regulatory_context("client_0", …)` reads ONLY `tenant_client_0_docs`. Collections are physically separate — no query can cross tenants. In the live app, `tenant_id` comes from API-key auth in `ingest.py` (`alert.tenant_id`).

---

6. Config & secrets

`backend/app/config.py` (RAG keys): `GROQ_MODEL`, `EMBEDDING_PROVIDER`, `LOCAL_EMBEDDING_MODEL`, `CHROMA_PERSIST_DIR`, `RAG_TOP_K_CHUNKS`, `MAX_UPLOAD_FILE_SIZE_MB`. `backend/.env` (gitignored) holds the real `GROQ_API_KEY` + overrides.

---

7. Test data & the DUMMY client (`client_0`)

The dummy/test client is named **`client_0`**; real clients use their tenant public id (`TEN-xxxx`).

UNIFIED per-client storage (one root, used by BOTH the live system and the eval):

```
backend/storage/clients/<client_id>/
├── policy.pdf          # raw AML policy (uploaded live, or built for client_0) —
NOT in DB
├── alerts/*.json       # {"raw_payload": {...}} — client_0 fixtures, or DB
exports for real clients
├── sar/*.pdf           # generated SAR PDFs (live output)
└── eval.json           # IR answer key (client_0 ONLY)
```

- `client_0` is the offline test bench — built by `scripts/seed_testing.py` (which calls `scripts/build_policy.py` to generate the synthetic policy).
- Real clients (`TEN-xxxx`) get their folder created **live** by the upload endpoint (`policy.pdf` + chunks into Chroma). Their transactions go to the Postgres `alerts` table — NOT mirrored to disk. To eval them, export with `scripts/export_alerts.py`.
- All generation demo scripts use `client_0`'s `policy.pdf` + an alert.

---

8. How to run

All commands are in **§0 (Quick command reference)** at the top. Rule of thumb:

- Always run from the repo root.
- `eval/*.py` read `backend/storage/clients/`; an optional client id narrows to one. IR needs an `eval.json` (client_0); RAGAS needs none (any client).
- `scripts/` generation = TEST-ONLY (`client_0`); eval (`ir_metrics`, `ragas_eval`) need no key except `ragas_eval` (Groq judge); generation needs the Groq key.

---

9. Reset tooling — `scripts/reset.py` (the reset button)

One command to get a clean, known state at any point:

Command	Effect
`python scripts/reset.py`	**Wipe runtime only:** `outputs/` + all Chroma stores (repo + temp). Client data KEPT. Safe, re-runnable any time.
`python scripts/reset.py --seed`	Wipe runtime, then **rebuild client_0** under `backend/storage/clients/client_0/` (policy + alerts + eval.json).
`python scripts/reset.py --barebone`	Also deletes the **dummy `client_0`** only — REAL clients are KEPT (safe default).
`python scripts/reset.py --barebone --seed`	Wipe client_0, then rebuild it from scratch.
`python scripts/reset.py --barebone --all-clients`	**DANGER:** deletes EVERY client folder incl. real clients' uploaded `policy.pdf` + SARs (their DB + Chroma survive).

`scripts/seed_testing.py` is the single source of the `client_0` fixtures (it embeds the 5 eval alerts + `eval.json` and calls `build_policy.py`), so `--barebone` is always reversible for `client_0` with `--seed`. Never touches: `backend/app`, `scripts/`, `*.md`, or PostgreSQL (`backend/clear_db.py` for SQL).

> The vector store rebuilds automatically the next time you run any script, so a plain `python scripts/reset.py` mid-development is always safe.

---

10. Evaluation — `eval/` (deterministic, NO LLM)

Per-client, because the answer key (section numbers) is document-specific. All clients live under `backend/storage/clients/<client_id>/`. IR scores any client that has an `eval.json` — in practice `client_0`.

```
backend/storage/clients/client_0/
├── eval.json     # schema_preset + rule_to_section (ANSWER KEY) + always_relevant
├── policy.pdf    # this client's doc → indexed into a temp collection
└── alerts/*.json # {"raw_payload": {...}} — this client's test alerts
```

Ground truth = stored map + live rule firing. The `rule_to_section` map (in `eval.json`) is stored; per-alert relevance is derived at runtime by running the real rule engine: `relevant = {section of each fired rule} ∪ {5.1}`. No hand-labeling.

Metrics (pure arithmetic):

```
precision@k = (# relevant in top-k) / k
recall@k    = (# relevant in top-k) / (# relevant)
ndcg@k      = DCG/IDCG,  DCG = Σ rel_i / log2(i+2)
mrr         = 1 / (rank of first relevant)
```

Run: `python eval/ir_metrics.py [client_id]`. It loops every tenant, prints per-alert + per-tenant means + an OVERALL (mean across tenants), and saves `outputs/ir_metrics.json`.

Score a real client's retrieval later (optional, manual): add an `eval.json` to `backend/storage/clients/<id>/` whose `rule_to_section` uses THEIR section numbers (plus their `policy.pdf` + some `alerts/*.json`). Run `python eval/ir_metrics.py <id>` — it auto-discovers any client with an `eval.json` and scores against its own answer key. (For generation quality, RAGAS needs no answer key — see §0.)

---

11. Results (client_0, 5 alerts)

Metric	Value	Meaning
Recall@8	**1.000**	finds every relevant section
MRR	**1.000**	#1 hit always relevant
nDCG@8	**0.955**	relevant chunks ranked high
nDCG@3	0.765	a plausible-but-unlabeled section (5.3) sometimes enters top-3
P@8	0.425	capped (only 3-4 relevant per alert; ignore as a signal)

Verdict: retrieval is near-optimal → reranking/hybrid NOT needed yet. Re-run whenever the corpus changes; let numbers decide future layers.

---

12. Key decisions (at a glance — full reasoning in §17)

- Encoder: **bge-small** local (SBERT bi-encoder), 384-dim, cosine, no PII egress.
- Vector store: **ChromaDB** local, per-tenant collections.
- Chunking: ~350 tok, 60 overlap, sentence-aligned, heading sections, context prefix.
- Queries: **rule-derived sub-queries** (not the JSON); chunks injected after `<<END DATA>>`.
- Eval-first: IR (retrieval) + RAGAS (generation) → **no reranking/hybrid yet**.
- **Mock vs real** split; generation fixed to one input until real clients exist.

---

13. Anomaly audit (fixed 2026-06-16)

- Hardcoded footer skip → generic running-header/footer detection (multi-client).
- `MAX_CHUNK_HARD` unused → over-long sentences hard-split (no silent truncation).
- Cover/front-matter (bank name, title) → dropped before first numbered heading.
- Heading detection tightened (non-numbered must be ≤60 chars).
- Dummy client renamed `client_1` → `client_0`; all scripts use `client_0`.
- After fixes: 28 chunks (was 29), metrics unchanged → removed chunks were noise.

---

14. Known issues (deferred, cosmetic)

- **SAR PDF table clipping:** long cells in `render_pdf` don't word-wrap → clipped. Fix = wrap cell strings in `Paragraph()`. Data correct; rendering only.
- **LLM structured-output shape** varied; mitigated by pinned JSON schema + `normalize_structured`.
- Eval uses client `display_name` in the context-line; demo scripts use a fixed title. Both retrieve near-perfectly; minor.

---

15. Frontend (light — see existing specs)

The React/Vite frontend (`frontend/`) and its specs (`LandingPageSpec.md`, `MVP.md`) are separate. RAG touch-points the frontend will need: a document-upload page (Phase-1 trigger) and the SAR review/approve workspace (reads the canonical SAR record; approve fires goAML serialize → webhook). Not built yet.

---

16. Testing vs Production — what's code, data, and API

This is the key mental model. There are TWO different worlds:

A. Offline tooling (eval + demo scripts) — folder-driven:

- Everything reads `backend/storage/clients/<client_id>/` — the SAME root the live system writes. `client_0` is the dummy; real clients use `TEN-xxxx`.
- `python eval/ir_metrics.py` scores any client with an `eval.json` (= `client_0`). `ragas_eval.py <id>` scores any client (no answer key).
- The demo scripts (`scripts/*.py`) are TEST-ONLY and use `client_0`. They are not how real clients are served.

B. The live API (real production path) — NOT folder/command-driven:

- A running FastAPI app serves ALL real tenants at once. The tenant is decided by the **API key** on each request (`ingest.py` → `alert.tenant_id`).
- A client's **policy.pdf** is stored under `backend/storage/clients/<public_id>/` (uploaded live) + chunks in per-tenant Chroma. Their **alerts** live in the Postgres `alerts` table (NOT mirrored to disk).
- "Test vs prod" here means separate DEPLOYMENTS/databases, NOT a folder.

The RAG code itself is environment-agnostic. `embeddings.py`, `chroma_client.py`, `document_ingestion_service.py`, `rag_retrieval_service.py`, `llm_agent.py` behave identically for any tenant, mock or real. Embeddings + Chroma are LOCAL; Groq works (watch rate limits at high volume).

So: *whatever the pipeline does on the dummy doc, it does identically on a real doc.* The eval just reads each client's folder.

To IR-score a real client you author their `eval.json` `rule_to_section` with THEIR section numbers (manual, optional). RAGAS needs none — just export their alerts (`scripts/export_alerts.py`).

16a. Generation eval — RAGAS (DONE, native implementation)

`eval/ragas_eval.py` measures GENERATION quality (what IR metrics can't):

- **Faithfulness** — extract atomic claims from the SAR (Groq), check each against the grounding = (transaction data + retrieved policy chunks). Score = supported/total.
- **Answer Relevancy** — generate questions the SAR answers (Groq), embed them + the real question (bge), score = mean cosine similarity.

Result (client_0, 5 alerts): **Faithfulness ≈ 0.81, Answer Relevancy ≈ 0.70** → generation is grounded and on-topic. Combined with IR (Recall@8=1.0), the whole pipeline is validated → no reranking/hybrid justified yet.

Why native, not the `ragas` package: ragas 0.4.x imports a `langchain_community` path removed in LangChain 1.x → won't import on this env. We implement the same two metrics directly with Groq + bge (more robust, same methodology).

Gotchas (encoded in the script): create the Groq client LAZILY (module-level creation before torch loads segfaults); set `OMP_NUM_THREADS=1`, `TOKENIZERS_PARALLELISM=false`, `KMP_DUPLICATE_LIB_OK=TRUE` (the ragas/langchain install bumped numpy/torch → intermittent segfaults without these); faithfulness grounding MUST include the transaction data, not just policy chunks (else true transaction facts score as "unsupported").

16b. Live-API status and remaining work

Live-API plumbing — now BUILT (verified against the routers; the earlier "deferred" note here was stale):

- `documents.py` upload router — a real client uploads their policy via `POST /api/v1/documents/upload`; it is stored to the client folder and indexed into THEIR Chroma collection (re-upload resets the collection first). Indexing no longer depends on scripts. (A dedicated `TenantDocument` audit table is still optional/deferred — the upload writes to folder + Chroma, not a DB row.)
- `retrieve_regulatory_context` is wired into `ingest.py` `process_alert_background`: the live alert path runs RAG and passes the retrieved chunks into `generate_sar()`, so policy-cited SARs fire on real alerts through the API. A RAG failure degrades to no-context generation and never fails the alert.
- Real approval → goAML → webhook delivery is live: `POST /api/v1/alerts/queue/{id}/approve` rehydrates PII, builds the goAML STR, renders the PDF, and POSTs the HMAC-signed webhook to the bank.
- The live path is exercised end-to-end by `scripts/simulator_client.py` and `scripts/verify_stack.py`.

Generation eval (RAGAS): DONE — implemented natively (Groq judge + bge); see §16a. Faithfulness ≈ 0.81, Answer Relevancy ≈ 0.70.

Genuinely remaining (not blockers; see also `STATUS_AND_MOCKBANK.md` §3):

- LLM-outage retry queue (Celery/Redis); background (async) webhook delivery.
- Optional `TenantDocument` table for richer upload audit.
- Optional per-client `eval.json` to run IR (not just RAGAS) on a real tenant.

---

17. Design journey & reasoning (how we got here, and why)

This captures the *thinking* — so a future agent understands not just what exists but why each choice was made.

The build order (chronological):

1. Read the goal (cite the tenant's real AML policy, not the LLM's general knowledge) and confirmed nothing existed yet.
2. Locked the encoder/similarity/chunking decisions FIRST (they ripple through every file), before writing code.
3. Built a *synthetic* test policy we control (so we know exactly what retrieval *should* return) with thresholds matching the rule engine.
4. Wrote a mock alert, validated it against the REAL rule engine (not assumed).
5. Built the 4 services + proved retrieval with a smoke test (no DB, no Groq) — isolate failures to one layer.
6. Added generation (Groq) → saw a real cited SAR; fixed a JSON-parse bug found.
7. Built the full demo (→ goAML) for colleagues; pinned the JSON schema after seeing the LLM's output shape vary.
8. Built deterministic IR eval *because* the question arose: "do we need reranking/hybrid?" — we wanted numbers, not vibes.
9. Made eval per-tenant (real clients each have a different policy doc).
10. Split mock vs real data into `testing/` and `production/` + a one-command seed/reset, so the dummy env is reproducible and wipeable.

The conceptual reasoning (the "why" behind the design):

- **Bi-encoder, not cross-encoder.** A bi-encoder encodes chunk and query separately → store chunk vectors once → cheap cosine (`queries + chunks`). A cross-encoder scores every pair live (`queries × chunks`) and can't back a vector DB. (A reranker IS a cross-encoder on the top ~50 — a later option.)
- **bge/SBERT, not raw BERT.** Raw BERT's pooled vectors are anisotropic (collapsed cone) → cosine is meaningless. SBERT/bge is BERT *fine-tuned* so cosine reflects meaning. bge IS a BERT, trained the right way. Same model encodes both sides (mandatory — else vectors live in different spaces).
- **We embed rule-derived sub-queries, NOT the JSON.** Masked tokens + numbers are semantic noise; the policy contains none of them. The compliance rules already decided which fields matter, so each fired rule → a regulation-worded query.
- **Contextual chunking.** Prepending `[Context: doc - section]` makes a chunk findable even when its body lacks the query keyword. (The prefix text is part of the embedding — changing its wording shifts rankings slightly.)
- **Inject chunks AFTER `<<END DATA>>`.** Chunks are trusted policy text; keeping them outside the untrusted-payload fence preserves prompt-injection safety.
- **Eval-first, then decide.** Don't add reranking/hybrid/CRAG speculatively. IR metrics (exact, no LLM, because we own the labels) showed retrieval is near-optimal (Recall@8=1.0, nDCG@8≈0.97) → those

layers aren't justified yet. RAGAS (LLM-judge) comes next for the GENERATION side, which IR can't measure.

- **PII never reaches RAG.** Masking happens before retrieval; the policy has no PII; Groq sees only tokens. Rehydration happens only at officer approval. So a TEE isn't load-bearing for RAG (it would matter only for the masker/PDF side).
- **Mock vs real separation.** `testing/` (regeneratable mock) vs `production/` (empty, real) keeps synthetic data from ever mixing with real clients — standard practice in finance where you can't test on real customer data. `seed_testing.py` embeds the fixtures so the whole mock env is reproducible from code.

Lessons / gotchas encountered:

- LLM wraps JSON in ``````` <persist_dir>/ ├── chroma.sqlite3 # documents + metadata + ids + catalog └── / # one folder per collection ├── data_level0.bin # 384-dim vectors + HNSW graph nodes ├── header.bin / length.bin └── link_lists.bin # HNSW graph edges (fast approx-NN search)

```
Default path = `backend/chroma_data/` (gitignored). Test scripts override it to a
temp dir so runs don't pollute the repo. Production: point `CHROMA_PERSIST_DIR` at
a stable path, or move to Chroma server / pgvector — only that setting changes.

---

## 5. Multi-tenant isolation

`tenant_id` threads through: `index_document("client_0", …)` →
`tenant_client_0_docs`;
`retrieve_regulatory_context("client_0", …)` reads ONLY `tenant_client_0_docs`.
Collections are physically separate — no query can cross tenants. In the live app,
`tenant_id` comes from API-key auth in `ingest.py` (`alert.tenant_id`).

---

## 6. Config & secrets

`backend/app/config.py` (RAG keys): `GROQ_MODEL`, `EMBEDDING_PROVIDER`,
`LOCAL_EMBEDDING_MODEL`, `CHROMA_PERSIST_DIR`, `RAG_TOP_K_CHUNKS`,
`MAX_UPLOAD_FILE_SIZE_MB`.
`backend/.env` (gitignored) holds the real `GROQ_API_KEY` + overrides.

---

## 7. Test data & the DUMMY client (`client_0`)

The dummy/test client is named **`client_0`**; real clients use their tenant
public id (`TEN-xxxx`).

**UNIFIED per-client storage (one root, used by BOTH the live system and the
eval):**
```

backend/storage/clients/<client_id>/ ├── policy.pdf # raw AML policy (uploaded live, or built for client_0) — NOT in DB ├── alerts/*.json # {"raw_payload": {...}} — client_0 fixtures, or DB exports for real clients ├── sar/*.pdf # generated SAR PDFs (live output) └── eval.json # IR answer key (client_0 ONLY)

```
- `client_0` is the offline test bench — built by `scripts/seed_testing.py`
  (which calls `scripts/build_policy.py` to generate the synthetic policy).
- Real clients (`TEN-xxxx`) get their folder created **live** by the upload
endpoint
  (`policy.pdf` + chunks into Chroma). Their transactions go to the Postgres
`alerts`
  table — NOT mirrored to disk. To eval them, export with
`scripts/export_alerts.py`.
- All generation demo scripts use `client_0`'s `policy.pdf` + an alert.

---

## 8. How to run

All commands are in **§0 (Quick command reference)** at the top. Rule of thumb:
- Always run from the repo root.
- `eval/*.py` read `backend/storage/clients/`; an optional client id narrows to
one.
  IR needs an `eval.json` (client_0); RAGAS needs none (any client).
- `scripts/` generation = TEST-ONLY (`client_0`); eval (`ir_metrics`,
`ragas_eval`)
  need no key except `ragas_eval` (Groq judge); generation needs the Groq key.

---

## 9. Reset tooling — `scripts/reset.py` (the reset button)

One command to get a clean, known state at any point:

| Command | Effect |
|---------|--------|
| `python scripts/reset.py` | **Wipe runtime only:** `outputs/` + all Chroma
stores (repo + temp). Client data KEPT. Safe, re-runnable any time. |
| `python scripts/reset.py --seed` | Wipe runtime, then **rebuild client_0** under
`backend/storage/clients/client_0/` (policy + alerts + eval.json). |
| `python scripts/reset.py --barebone` | Also deletes the **dummy `client_0`**
only — REAL clients are KEPT (safe default). |
| `python scripts/reset.py --barebone --seed` | Wipe client_0, then rebuild it
from scratch. |
| `python scripts/reset.py --barebone --all-clients` | **DANGER:** deletes EVERY
client folder incl. real clients' uploaded `policy.pdf` + SARs (their DB + Chroma
survive). |

`scripts/seed_testing.py` is the single source of the `client_0` fixtures (it
embeds
the 5 eval alerts + `eval.json` and calls `build_policy.py`), so `--barebone` is
always reversible for `client_0` with `--seed`. Never touches: `backend/app`,
```

```
`scripts/`, `*.md`, or PostgreSQL (`backend/clear_db.py` for SQL).

> The vector store rebuilds automatically the next time you run any script, so a
> plain `python scripts/reset.py` mid-development is always safe.

---

## 10. Evaluation — `eval/` (deterministic, NO LLM)

Per-client, because the answer key (section numbers) is document-specific. All
clients live under `backend/storage/clients/<client_id>/`. IR scores any client
that
has an `eval.json` — in practice `client_0`.
```

backend/storage/clients/client_0/ ├── eval.json # schema_preset + rule_to_section (ANSWER KEY) + always_relevant ├── policy.pdf # this client's doc → indexed into a temp collection └── alerts/*.json # {"raw_payload": {...}} — this client's test alerts

```


**Ground truth = stored map + live rule firing.** The `rule_to_section` map (in
`eval.json`) is stored; per-alert relevance is derived at runtime by running the
real rule engine: `relevant = {section of each fired rule} ∪ {5.1}`. No hand-
labeling.

**Metrics (pure arithmetic):**
```python
precision@k = (# relevant in top-k) / k
recall@k    = (# relevant in top-k) / (# relevant)
ndcg@k      = DCG/IDCG,  DCG = Σ rel_i / log2(i+2)
mrr         = 1 / (rank of first relevant)
```
```

Run: `python eval/ir_metrics.py [client_id]`. It loops every tenant, prints per-alert + per-tenant means + an OVERALL (mean across tenants), and saves `outputs/ir_metrics.json`.

Score a real client's retrieval later (optional, manual): add an `eval.json` to `backend/storage/clients/<id>/` whose `rule_to_section` uses THEIR section numbers (plus their `policy.pdf` + some `alerts/*.json`). Run `python eval/ir_metrics.py <id>` — it auto-discovers any client with an `eval.json` and scores against its own answer key. (For generation quality, RAGAS needs no answer key — see §0.)

---

11. Results (client_0, 5 alerts)

Metric	Value	Meaning
Recall@8	**1.000**	finds every relevant section
MRR	**1.000**	#1 hit always relevant
nDCG@8	**0.955**	relevant chunks ranked high

Metric	Value	Meaning
nDCG@3	0.765	a plausible-but-unlabeled section (5.3) sometimes enters top-3
P@8	0.425	capped (only 3-4 relevant per alert; ignore as a signal)

Verdict: retrieval is near-optimal → reranking/hybrid NOT needed yet. Re-run whenever the corpus changes; let numbers decide future layers.

---

12. Key decisions (at a glance — full reasoning in §17)

- Encoder: **bge-small** local (SBERT bi-encoder), 384-dim, cosine, no PII egress.
- Vector store: **ChromaDB** local, per-tenant collections.
- Chunking: ~350 tok, 60 overlap, sentence-aligned, heading sections, context prefix.
- Queries: **rule-derived sub-queries** (not the JSON); chunks injected after `<<END DATA>>`.
- Eval-first: IR (retrieval) + RAGAS (generation) → **no reranking/hybrid yet**.
- **Mock vs real** split; generation fixed to one input until real clients exist.

---

13. Anomaly audit (fixed 2026-06-16)

- Hardcoded footer skip → generic running-header/footer detection (multi-client).
- `MAX_CHUNK_HARD` unused → over-long sentences hard-split (no silent truncation).
- Cover/front-matter (bank name, title) → dropped before first numbered heading.
- Heading detection tightened (non-numbered must be ≤60 chars).
- Dummy client renamed `client_1` → `client_0`; all scripts use `client_0`.
- After fixes: 28 chunks (was 29), metrics unchanged → removed chunks were noise.

---

14. Known issues (deferred, cosmetic)

- **SAR PDF table clipping:** long cells in `render_pdf` don't word-wrap → clipped. Fix = wrap cell strings in `Paragraph()`. Data correct; rendering only.
- **LLM structured-output shape** varied; mitigated by pinned JSON schema + `normalize_structured`.
- Eval uses client `display_name` in the context-line; demo scripts use a fixed title. Both retrieve near-perfectly; minor.

---

15. Frontend (light — see existing specs)

The React/Vite frontend (`frontend/`) and its specs (`LandingPageSpec.md`, `MVP.md`) are separate. RAG touch-points the frontend will need: a document-upload page (Phase-1 trigger) and the SAR review/approve workspace (reads the canonical SAR record; approve fires goAML serialize → webhook). Not built yet.

---

16. Testing vs Production — what's code, data, and API

This is the key mental model. There are TWO different worlds:

A. Offline tooling (eval + demo scripts) — folder-driven:

- Everything reads `backend/storage/clients/<client_id>/` — the SAME root the live system writes. `client_0` is the dummy; real clients use `TEN-xxxx`.
- `python eval/ir_metrics.py` scores any client with an `eval.json` (= `client_0`). `ragas_eval.py <id>` scores any client (no answer key).
- The demo scripts (`scripts/*.py`) are TEST-ONLY and use `client_0`. They are not how real clients are served.

B. The live API (real production path) — NOT folder/command-driven:

- A running FastAPI app serves ALL real tenants at once. The tenant is decided by the **API key** on each request (`ingest.py` → `alert.tenant_id`).
- A client's **policy.pdf** is stored under `backend/storage/clients/<public_id>/` (uploaded live) + chunks in per-tenant Chroma. Their **alerts** live in the Postgres `alerts` table (NOT mirrored to disk).
- "Test vs prod" here means separate DEPLOYMENTS/databases, NOT a folder.

The RAG code itself is environment-agnostic. `embeddings.py`, `chroma_client.py`, `document_ingestion_service.py`, `rag_retrieval_service.py`, `llm_agent.py` behave identically for any tenant, mock or real. Embeddings + Chroma are LOCAL; Groq works (watch rate limits at high volume).

So: *whatever the pipeline does on the dummy doc, it does identically on a real doc.* The eval just reads each client's folder.

To IR-score a real client you author their `eval.json` `rule_to_section` with THEIR section numbers (manual, optional). RAGAS needs none — just export their alerts (`scripts/export_alerts.py`).

16a. Generation eval — RAGAS (DONE, native implementation)

`eval/ragas_eval.py` measures GENERATION quality (what IR metrics can't):

- **Faithfulness** — extract atomic claims from the SAR (Groq), check each against the grounding = (transaction data + retrieved policy chunks). Score = supported/total.
- **Answer Relevancy** — generate questions the SAR answers (Groq), embed them + the real question (bge), score = mean cosine similarity.

Result (client_0, 5 alerts): **Faithfulness ≈ 0.81, Answer Relevancy ≈ 0.70** → generation is grounded and on-topic. Combined with IR (Recall@8=1.0), the whole pipeline is validated → no reranking/hybrid justified yet.

Why native, not the `ragas` package: ragas 0.4.x imports a `langchain_community` path removed in LangChain 1.x → won't import on this env. We implement the same two metrics directly with Groq + bge (more robust, same methodology).

Gotchas (encoded in the script): create the Groq client LAZILY (module-level creation before torch loads segfaults); set `OMP_NUM_THREADS=1`, `TOKENIZERS_PARALLELISM=false`, `KMP_DUPLICATE_LIB_OK=TRUE` (the ragas/langchain install bumped numpy/torch → intermittent segfaults without these); faithfulness grounding MUST include the transaction data, not just policy chunks (else true transaction facts score as "unsupported").

16b. Live-API status and remaining work

Live-API plumbing — now BUILT (verified against the routers; the earlier "deferred" note here was stale):

- `documents.py` upload router — a real client uploads their policy via `POST /api/v1/documents/upload`; it is stored to the client folder and indexed into THEIR Chroma collection (re-upload resets the collection first). Indexing no longer depends on scripts. (A dedicated `TenantDocument` audit table is still optional/deferred — the upload writes to folder + Chroma, not a DB row.)
- `retrieve_regulatory_context` is wired into `ingest.py` `process_alert_background`: the live alert path runs RAG and passes the retrieved chunks into `generate_sar()`, so policy-cited SARs fire on real alerts through the API. A RAG failure degrades to no-context generation and never fails the alert.
- Real approval → goAML → webhook delivery is live: `POST /api/v1/alerts/queue/{id}/approve` rehydrates PII, builds the goAML STR, renders the PDF, and POSTs the HMAC-signed webhook to the bank.
- The live path is exercised end-to-end by `scripts/simulator_client.py` and `scripts/verify_stack.py`.

Generation eval (RAGAS): DONE — implemented natively (Groq judge + bge); see §16a. Faithfulness ≈ 0.81, Answer Relevancy ≈ 0.70.

Genuinely remaining (not blockers; see also `STATUS_AND_MOCKBANK.md` §3):

- LLM-outage retry queue (Celery/Redis); background (async) webhook delivery.
- Optional `TenantDocument` table for richer upload audit.
- Optional per-client `eval.json` to run IR (not just RAGAS) on a real tenant.

---

17. Design journey & reasoning (how we got here, and why)

This captures the *thinking* — so a future agent understands not just what exists but why each choice was made.

The build order (chronological):

1. Read the goal (cite the tenant's real AML policy, not the LLM's general knowledge) and confirmed nothing existed yet.
2. Locked the encoder/similarity/chunking decisions FIRST (they ripple through every file), before writing code.
3. Built a *synthetic* test policy we control (so we know exactly what retrieval *should* return) with thresholds matching the rule engine.
4. Wrote a mock alert, validated it against the REAL rule engine (not assumed).
5. Built the 4 services + proved retrieval with a smoke test (no DB, no Groq) — isolate failures to one layer.
6. Added generation (Groq) → saw a real cited SAR; fixed a JSON-parse bug found.
7. Built the full demo (→ goAML) for colleagues; pinned the JSON schema after seeing the LLM's output shape vary.
8. Built deterministic IR eval *because* the question arose: "do we need reranking/hybrid?" — we wanted numbers, not vibes.
9. Made eval per-tenant (real clients each have a different policy doc).
10. Split mock vs real data into `testing/` and `production/` + a one-command seed/reset, so the dummy env is reproducible and wipeable.

The conceptual reasoning (the "why" behind the design):

- **Bi-encoder, not cross-encoder.** A bi-encoder encodes chunk and query separately → store chunk vectors once → cheap cosine (`queries + chunks`). A cross-encoder scores every pair live (`queries × chunks`) and can't back a vector DB. (A reranker IS a cross-encoder on the top ~50 — a later option.)
- **bge/SBERT, not raw BERT.** Raw BERT's pooled vectors are anisotropic (collapsed cone) → cosine is meaningless. SBERT/bge is BERT *fine-tuned* so cosine reflects meaning. bge IS a BERT, trained the right way. Same model encodes both sides (mandatory — else vectors live in different spaces).
- **We embed rule-derived sub-queries, NOT the JSON.** Masked tokens + numbers are semantic noise; the policy contains none of them. The compliance rules already decided which fields matter, so each fired rule → a regulation-worded query.
- **Contextual chunking.** Prepending `[Context: doc - section]` makes a chunk findable even when its body lacks the query keyword. (The prefix text is part of the embedding — changing its wording shifts rankings slightly.)
- **Inject chunks AFTER `<<END DATA>>`.** Chunks are trusted policy text; keeping them outside the untrusted-payload fence preserves prompt-injection safety.
- **Eval-first, then decide.** Don't add reranking/hybrid/CRAG speculatively. IR metrics (exact, no LLM, because we own the labels) showed retrieval is near-optimal (Recall@8=1.0, nDCG@8≈0.97) → those layers aren't justified yet. RAGAS (LLM-judge) comes next for the GENERATION side, which IR can't measure.
- **PII never reaches RAG.** Masking happens before retrieval; the policy has no PII; Groq sees only tokens. Rehydration happens only at officer approval. So a TEE isn't load-bearing for RAG (it would matter only for the masker/PDF side).
- **Mock vs real separation.** `testing/` (regeneratable mock) vs `production/` (empty, real) keeps synthetic data from ever mixing with real clients — standard practice in finance where you can't test on real customer data. `seed_testing.py` embeds the fixtures so the whole mock env is reproducible from code.

Lessons / gotchas encountered:

- LLM wraps JSON in ```json fences → parser must strip them and never discard the narrative on a JSON failure.
- LLM structured-output shape varies → pin the schema in the prompt + normalize on approval.
- Hardcoding one client's footer text breaks multi-client → detect running headers/footers generically.
- An unused `MAX_CHUNK_HARD` meant a giant sentence could silently truncate → hard-split.
- The context-line `doc_title` affects embeddings → eval vs demo can differ slightly.

---

18. Change log

- 2026-06-16: Built vanilla RAG (Phases 1-3), demo, per-tenant IR eval. Anomaly audit + fixes. Renamed dummy client to `client_0`. Added `scripts/reset.py`. Consolidated docs into this file (replaced RAG_TECHNICAL_NOTES / RAG_WALKTHROUGH / PRE_RAGAS).
- 2026-06-16 (later 4): RAGAS done — `eval/ragas_eval.py` (native, Groq judge + bge). Faithfulness≈0.81, Answer Relevancy≈0.70. ragas pip pkg incompatible w/ LangChain 1.x; segfault fixes (lazy Groq + OMP/tokenizer env guards); faithfulness grounds on transaction data + chunks. Pipeline fully validated; no reranking/hybrid needed.
- 2026-06-16 (later 3): User deleted the older root docs (BuildOrder/LandingPageSpec/MVP/PRD/RAG_HANDOFF/README). Made THIS file fully self-sufficient: added §17 "Design journey &

reasoning" (the thinking + lessons), ensured §0 has every command. Remaining root md: APISpec, DatabaseSchema, RAG_MASTER, explanation.

- 2026-06-16 (later 2): Added §0 quick command reference and §16 "Testing vs Production" mental model (offline tooling is folder/flag-driven; the live API is tenant-keyed, not folder-driven; RAG code is environment-agnostic).
- 2026-06-16 (later): Split mock vs real data into `testing/` (regeneratable mock) and `production/` (empty, real). Moved policy→`testing/corpus`, input→`testing/inputs`, eval client_0→`testing/clients`; removed `aml_corpus/`, `mock_inputs/`, `eval/tenants/`. Added `scripts/seed_testing.py` (embeds all mock fixtures, fully regenerates). `reset.py --seed` now rebuilds the whole mock env; `--barebone` wipes `testing/` data. `eval/ir_metrics.py --prod` evaluates `production/` clients. Verified full wipe→regenerate→eval cycle.
- 2026-06-17: Added §19 "Full walkthrough" (read-it-start-to-finish narrative: goal → concepts → code → eval). RAGAS made per-client + `--prod`. Slimmed §8/§12 to remove duplication.
- 2026-06-19: **Unified per-client storage.** Replaced the `testing/` vs `production/` split with ONE root `backend/storage/clients/<client_id>/` (policy.pdf + alerts/ + sar/ + eval.json), the SAME place the live upload writes and the eval reads. Decisions: policy PDF = bank self-serve upload → folder + Chroma (NOT in DB); alerts → Postgres `alerts` table (removed the disk-mirror from ingest.py); IR needs an `eval.json` so it runs on `client_0` only; RAGAS is reference-free so it runs on any client. RAGAS lost `--prod`/`config.json` (reads the unified folders, schema_preset from eval.json or default). Added `scripts/export_alerts.py` (DB alerts → folder JSON for real-client RAGAS). Moved `build_policy.py` → `scripts/`. DELETED `testing/`, `production/`, `backend/storage/{policies,sar}`. Verified IR Recall@8=1.0 + smoke oracle on the new layout.
- 2026-06-22: Documentation accuracy pass (no code changes). Corrected §16b, which still described the live-API plumbing as "not built yet": the `documents.py` upload router, RAG wired into `ingest.py` `process_alert_background`, and approval → goAML → webhook delivery are all built and verified against the routers. RAGAS is done (§16a), not pending. Also normalised the doc set: added tables of contents, removed emoji markers, and fixed minor syntax. `AEGIS_KNOWLEDGE_BASE.md` remains the code-verified source of truth.

---

19. Full walkthrough (read start-to-finish: goal → concepts → code → eval)

A single narrative explainer. The earlier sections are the structured reference; this is the teaching version for someone learning the whole thing cold.

1. The goal

Aegis writes SARs with Groq. Without RAG the LLM uses general knowledge and can't cite the tenant's policy. RAG fetches the relevant policy slices and feeds them to the LLM, so the SAR cites e.g. "Section 4.1" instead of vague prose. RAG = Retrieve relevant policy → Augment the prompt → Generate the cited SAR.

2. Core concepts

- **Embeddings:** text → 384-number vector; similar meaning → similar direction; compared by **cosine similarity** (angle; small angle = similar).
- **Bi-encoder vs cross-encoder:** a bi-encoder encodes chunk and query SEPARATELY → store chunk vectors once → cheap cosine (`chunks + queries`). A cross-encoder scores each (query,chunk) PAIR live

(`chunks × queries`) and can't back a vector DB. (A reranker is a cross-encoder on the top ~50 — a later option.)

- **BERT vs SBERT:** raw BERT vectors are anisotropic (cosine meaningless); bge is BERT fine-tuned so cosine reflects meaning. bge IS a BERT, trained right.
- **Asymmetric:** prefix the QUERY only ("Represent this sentence for searching relevant passages: "); chunks raw.
- **Contextual chunking:** prepend `[Context: doc - section]` to each chunk → it's findable even without the keyword.
- **Don't embed the JSON:** masked tokens/numbers are noise → embed rule-derived sub-queries instead.

3. The pipeline + code (3 phases)

Shared engine: `embeddings.py` (make vectors) + `chroma_client.py` (per-tenant store).

```
# embeddings.py — one model, loaded once, used by BOTH sides
@lru_cache(maxsize=1)
def _local_model():
    from sentence_transformers import SentenceTransformer
    return SentenceTransformer("BAAI/bge-small-en-v1.5")
def embed_documents(texts): return _local_model().encode(texts,
normalize_embeddings=True).tolist()
def embed_query(text): return _local_model().encode(["Represent this sentence for
searching relevant passages: "+text], normalize_embeddings=True)[0].tolist()

# chroma_client.py — per-tenant isolation + cosine
def get_tenant_collection(tid):
    return chromadb.PersistentClient(path=_PERSIST_DIR).get_or_create_collection(
        name=f"tenant_{tid}_docs", metadata={"hnsw:space": "cosine"})
```

Phase 1 (index) — document_ingestion_service.py: PyMuPDF parse (keep font size

- page) → split on headings → ~350-token sentence-aligned chunks w/ 60 overlap → prepend context line → embed → `collection.add(...)`. **Phase 2 (retrieve) — rag_retrieval_service.py:** fired rules → `RULE_TO_QUERY` sub-queries → embed each → `collection.query(n_results=5)` → merge/dedupe → top-8. **Phase 3 (generate) — llm_agent.py:** inject chunks AFTER `<<END DATA>>` (trusted text) → Groq → parse narrative + structured JSON → rehydrate PII at approval.

4. Where data lives

ChromaDB local files: `chroma.sqlite3` (docs+metadata) + per-collection HNSW binaries. No server. Tests use a temp dir.

5. Multi-tenant

`tenant_id` keys the collection name, so `index_document("client_0",…)` and `retrieve_regulatory_context("client_0",…)` only ever touch `tenant_client_0_docs`. No query crosses tenants. Live app gets `tenant_id` from the API key.

6. Evaluation (both halves)

- **Retrieval — ir_metrics.py (no LLM):** we own the labels (rule → section in `eval.json`), so Precision@k/Recall@k/nDCG/MRR are pure arithmetic. Result: Recall@8=1.0, nDCG@8=0.98, MRR=1.0.
- **Generation — ragas_eval.py (Groq judge + bge):** Faithfulness (Groq extracts claims, checks each vs transaction-data+chunks) = 0.81; Answer Relevancy (Groq makes questions from the SAR, bge cosine-compares to the real question) = 0.70.

7. Test infrastructure

ONE root `backend/storage/clients/<client_id>/` for every client (dummy = `client_0`, real = `TEN-xxxx`) — the same folders the live system writes and the eval reads. `scripts/reset.py` (+`--seed`/`--barebone`) and `scripts/seed_testing.py` make the `client_0` test bench fully wipeable and reproducible.

8. Outcome

Vanilla RAG was sufficient — proven by metrics, not assumed. No reranking/ hybrid/CRAG needed (retrieval perfect, generation grounded). Caveat: re-evaluate when real multi-doc corpora arrive. Remaining work = wire RAG into the live app (API endpoints + frontend); the RAG brain itself is done.

One line: embeddings make vectors → ingestion fills Chroma (P1) → retrieval cosine-searches it (P2) → llm_agent writes the cited SAR (P3) → ir_metrics proves retrieval + ragas_eval proves generation → all per-tenant, unified storage, resettable.

---

20. Security architecture & threat model

The backend is a decoupled FastAPI + SQLAlchemy + PostgreSQL app. Request lifecycle: routers (RBAC via `Depends`) → services (the "brain") → models (SQL tables).

The AML pipeline (`app/services/`, called from `routers/ingest.py`):

1. **Normalizer** (`schema_normalizer.py`) — proprietary bank JSON → standard Aegis fields using the tenant's own `IngestionSchema.field_map`.
2. **Rule engine** (`compliance_analyzer.py`) — deterministic typology rules → factual evidence (so the LLM doesn't hallucinate the suspicion).
3. **PII masker** (`pii_masker.py`) — replaces names/accounts with `<<TOKEN>>` before any external AI call; deterministic SHA-256 tokens so the same entity links across txns.
4. **AI agent** (`llm_agent.py`) — packages masked data + evidence + retrieved policy, sanitizes against prompt injection, asks Groq for the SAR, then rehydrates PII.

Implemented security patches (live):

- Timing-attack-safe auth (dummy bcrypt round on invalid tenant/email; constant-time API key compare via `secrets.compare_digest`).
- Pre-auth in-process rate limiting (sliding window, per tenant/IP, 429 + Retry-After, bounded to 10k buckets so random-key floods can't OOM).
- Idempotency / replay protection (`Idempotency-Key` or body SHA-256 + a DB `UniqueConstraint`; `IntegrityError` caught for the same-millisecond race).
- Isolated background DB sessions (own `SessionLocal`, closed in `finally`).
- Synthetic test alerts flagged `is_synthetic=True` (excluded from compliance metrics).
- LLM prompt-injection defense (`<<DATA>>` boundary markers + untrusted-data instruction).

- Webhook SSRF guard (DNS-resolves the URL, blocks private/loopback/link-local; dev carve-out).
- JWT token-type confusion prevention (`access` vs `refresh` in the payload).
- PII encryption at rest (`cryptography.fernet` over the `token_map`).
- Payload size caps (Content-Length + byte recount) to prevent OOM.
- Async non-blocking ingest (returns `202` fast, offloads LLM to a background task).

Threat-model status (was a TODO list; current state):

- **Addressed:** Prompt injection, Webhook SSRF, OOM payload cap, PII vault encryption, Replay attacks (idempotency), API-key rotation (refresh-token rotation invariant), DB concurrency (Postgres + IntegrityError handling; SQLite no longer used).
- **Deferred — LLM outage/rate-limit resilience** — a Groq outage makes the background SAR task fail; there's a safe degrade-to-no-context path, but no retry queue (Celery/Redis) yet. This is the main open hardening item for high volume.

---

21. Data schemas & the schema-faithful eval

There are **3 ingestion schemas** (`app/data/schema_presets.py`), each a different shape:

Schema	"customer name"	"amount"
`STANDARD_FINTECH`	`customer.full_name`	`txn.amount`
`SEBI_BROKER`	`client.name`	`trade.value`
`PAYMENT_GW`	`payer.name`	`txn.amount`

The live ingest normalizes each alert with the **tenant's own** schema (`ingest.py`) and stores the result as `normalized_payload` + `masked_payload` on the alert row.

Why the eval must respect this: the eval used to hardcode `STANDARD_FINTECH`, so a non-fintech client (e.g. a SEBI broker) would be scored against the wrong field map → all fields `None` → no rules fire → silently meaningless scores. **Fix:** `scripts/export_alerts.py` exports the DB-computed `normalized_payload`/`masked_payload`, and `ir_metrics.py`/`ragas_eval.py` use them (falling back to preset-normalize only for the `client_0` fixtures, which carry only `raw_payload`). The eval now scores exactly what live produced, for any schema.

---

22. Known edge cases & hardening (handled)

- **Eval on partial data:** a client folder missing `policy.pdf` or `alerts/` is skipped with a message; an alert with `normalized_payload=None` falls back to preset-normalize.
- **RAGAS without a Groq key:** `ragas_eval.py` exits with one clear message instead of erroring per alert. IR needs no key.
- **`reset.py --barebone` foot-gun:** now wipes ONLY the dummy `client_0`; the full nuke (incl. real clients' policies) requires the explicit `--all-clients` flag.
- **Super-admin policy upload:** a `SUPER_ADMIN` (no `tenant_id`) hitting the upload endpoint used to write to `clients/None/`; now returns `400` (must be a tenant user).

- **Non-UUID `sar_id`** on `GET /files/sar/{id}.pdf` now returns `404` (was a `500` from the Postgres UUID cast).
- **Webhook/PDF failures on approve** are non-fatal (wrapped) — approval still succeeds.
- **Curly-quote build break:** keep JS string quotes ASCII in the frontend (bit us once).
- **Deferred — Synchronous webhook on approve:** `_deliver_webhook` POSTs with an 8s timeout inline, so approving blocks up to 8s on a slow bank. Fine locally; background it for prod.